Introduzione: il frontend nel 2026

Corso di Frontend Development – Modulo 00

Federico Giuggioloni

ITS – Corso di Frontend Development

Anno formativo 2026/2027

Contenuto del modulo

1. Introduzione: il frontend nel 2026
1.1 Il corso
1.2 Il progetto: minicommerce
1.3 Il panorama del frontend nel 2026
1.4 Ambiente di lavoro
1.5 Il package manager
1.6 Ripasso rapido di TypeScript
1.7 L'API di minicommerce
1.8 Metodo di lavoro

Sommario del modulo

1. Introduzione: il frontend nel 2026
1.1 Il corso
1.2 Il progetto: minicommerce
1.3 Il panorama del frontend nel 2026
1.4 Ambiente di lavoro
1.5 Il package manager
1.6 Ripasso rapido di TypeScript
1.7 L'API di minicommerce
1.8 Metodo di lavoro

Benvenuti

- Corso: **Frontend Development** – secondo anno ITS.
- Il frontend non è più solo "il sito": è tutto ciò che l'utente vede e tocca.
- Obiettivo del corso: partire da **una** web app e portarla **ovunque**: browser, desktop, telefono, e capire quando invece conviene scrivere codice nativo.
- Non partiamo da zero: lavoriamo su **minicommerce**, un'applicazione vera e funzionante che vi consegno il primo giorno, con backend e frontend.
- Ogni lezione: teoria breve, poi laboratorio con il codice aperto.

Cosa sapete già (prerequisiti)

Dal primo anno

- HTML, CSS, JavaScript moderno
- Fondamenti di React (componenti, props, state, hook di base)
- Progettazione e sviluppo di una **API REST**
- Un primo contatto con **Capacitor / Ionic**
- Git di base

Cosa daremo per scontato

- Sapete leggere una risposta JSON
- Sapete cosa sono HTTP, verbi, status code
- Sapete usare il terminale
- Il backend di minicommerce è già scritto: in questo corso lo **consumiamo**, lo tocchiamo solo quando serve

Il programma in una slide

0. **Introduzione**: panorama, il progetto minicommerce, setup, ripasso TypeScript
1. **React**: da minicommerce a un'applicazione *completa*
2. **Desktop**: la stessa app diventa un eseguibile con Electron e Tauri
3. **Mobile**: responsive, PWA e app con Capacitor/Ionic
4. **SPA vs sito vetrina**: SSR/SSG con React, Astro, PHP e altri
5. **App native**: Windows, Linux, macOS, Android, iOS – incipit e vantaggi
6. **Qualità**: testing, accessibilità, performance, sicurezza
7. **Consegna**: CI/CD, deploy, monitoraggio
8. **Agenti di programmazione**: come lavorare (bene) con l'AI
9. **Progetto finale** e valutazione

Ripartizione indicativa delle ore

Modulo	Lezione (h)	Laboratorio (h)
0. Introduzione e setup	4	4
1. React: app completa	10	14
2. Desktop (Electron/Tauri)	4	6
3. Mobile (PWA/Capacitor)	6	8
4. Sito vetrina e SSR	6	6
5. App native	8	6
6. Qualità	4	6
7. Consegna e DevOps	3	3
8. Agenti AI	2	2
9. Progetto finale	2	16

Totale circa 120 ore. Se il monte ore è inferiore: tagliare prima i moduli 7 e 5 (parte Linux/macOS), poi il laboratorio del modulo 4.

minicommerce: il filo conduttore

- Un piccolo **e-commerce di articoli informatici**: PC, periferiche, smartphone, accessori.
- Due parti già scritte e funzionanti:
  - `backend/`: API REST con Express, SQLite e Drizzle, login JWT
  - `frontend/`: React 19 + Vite, Tailwind e shadcn/ui
- Tre schermate: **catalogo** pubblico, **login**, **area admin** per gestire gli articoli.
- Abbastanza semplice da tenere in testa, abbastanza ricca da toccare tutti i temi del corso.

Perché un'app sola

Cambiare piattaforma senza cambiare dominio permette di concentrarsi sulle *differenze* tra le piattaforme, non sul reinventare la logica ogni volta.

Alla fine del corso

La stessa minicommerce in 5 forme: web, desktop, PWA, app mobile, sito vetrina indicizzabile.

Le schermate, e i due tipi di utente

`/`	catalogo: ricerca, filtro per categoria, fascia di prezzo, ordinamento. Pubblico.
`/article/:id`	scheda di un articolo, con "aggiungi al carrello".
`/about`	pagina di presentazione.
`/login`	accesso; da qui si registra anche un nuovo cliente.
`/cart`	carrello del cliente e ordine (finto).
`/admin`	tabella articoli: crea, modifica, elimina.

Due ruoli, due mondi

Il campo `isAdmin` dell'utente divide l'applicazione in due:

- il **cliente** naviga il catalogo e riempie il carrello;
- l'**amministratore** gestisce gli articoli e *non* può usare il carrello (l'API gli risponde 403).

Tenetelo a mente: quasi ogni scelta del corso si spiega chiedendosi "per quale dei due?".

Com'è fatto il repository

```
minicommerce/
  README.md              # cosa fa, come si avvia, API, modello dati
  AGENTS.md              # convenzioni del progetto (lo useremo nel modulo 8)
  backend/
    src/server.ts        # Express 5: cors, json, rotte, gestore errori
    src/routes/          # auth.ts, categories.ts, products.ts
    src/middleware/      # auth.ts -> requireAdmin (verifica il JWT)
    src/db/schema.ts     # Drizzle: users, categories, products
    src/seed.ts          # dati demo + utente admin
  frontend/
    index.html
    vite.config.ts       # plugin react + tailwind, proxy /api -> :3001
    src/main.tsx         # BrowserRouter + AuthProvider
    src/App.tsx          # Header + <Routes> + <Toaster>
    src/pages/           # Catalog, Article, About, Login, Cart, Admin
    src/components/      # Header, ProductFormDialog, ui/ (shadcn)
    src/lib/             # api.ts (axios + tipi), auth.tsx (context)
```

Come si avvia

```
# 1. Backend (porta 3001)
cd backend
npm install
npm run db:push      # crea le tabelle SQLite (solo la prima volta)
npm run db:seed      # categorie, articoli demo e utente admin
npm run dev          # node --watch src/server.ts

# 2. Frontend (porta 5173), in un secondo terminale
cd frontend
npm install
npm run dev          # vite, con proxy /api -> http://localhost:3001

# App su http://localhost:5173
# Admin: admin@minicommerce.dev / admin123
```

- Serve **Node.js 24 o superiore**: il backend esegue TypeScript *nativamente*, senza compilazione.
- `npm run db:reset` ricrea il database da zero: comodo, ma cancella i vostri dati locali.

Lo stack di partenza

Parte	Tecnologia	Note
Linguaggio	TypeScript ovunque	backend e frontend
Backend	Node 24, Express 5	TypeScript eseguito nativamente
Database	SQLite + Drizzle ORM	un file, zero installazione
Auth	JWT + bcrypt	token di 8 ore, ruolo `isAdmin`
Build	Vite 8	dev server e bundle di produzione
UI	React 19, Tailwind v4	
Componenti	shadcn/ui su Radix	copiati dentro `src/components/ui/`
Icone, toast	lucide-react, sonner	
HTTP	Axios con interceptor	aggiunge il token a ogni richiesta
Routing	React Router 7	`BrowserRouter`
Lint	Oxlint	molto veloce, niente Prettier

Cosa c'è già e cosa manca

C'è, e funziona

- Catalogo con filtri, ricerca con debounce, ordinamento
- Login con JWT e sessione che sopravvive al refresh
- CRUD degli articoli con validazione del form
- Stato vuoto del catalogo, toast di errore
- Etichette, `aria-invalid`, `role="alert"` nel form

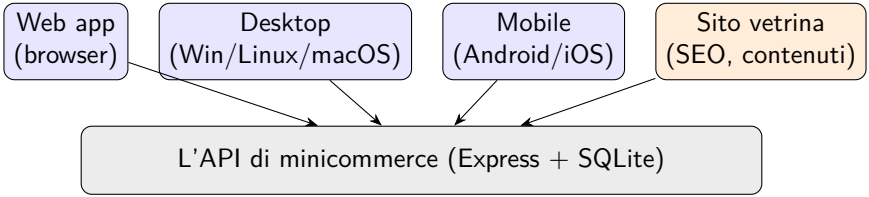
Manca, e lo aggiungeremo

- Nessuna cache: ogni pagina ricarica tutto da capo
- Filtri non nell'URL: non si condividono né si ricaricano
- Nessuno stato di errore in pagina, solo un toast
- Nessun test, nessuna CI, nessun deploy
- Niente offline, desktop o app mobile
- Il catalogo non è indicizzabile da Google

Il corso in una frase

Prendiamo un'app che funziona e la portiamo al livello di un prodotto vero, su cinque piattaforme.

Frontend: che cosa intendiamo



- Il frontend è ogni programma che parla con l'utente e con la vostra API.
- La domanda del corso: **quanto codice possiamo condividere** tra queste forme, e quando conviene **non** condividerlo?

Tre famiglie di tecnologie

Web-based
React, Vue, Svelte, Angular; impacchettate con Electron, Tauri, Capacitor, PWA.

- +Un solo codice
- +Team e strumenti web
- –Prestazioni e “feeling” non nativi

Cross-platform nativo
React Native/Expo, Flutter, .NET MAUI, Kotlin Multiplatform.

- +Widget nativi o quasi
- +Un solo linguaggio
- –Strato intermedio da mantenere

Nativo puro
SwiftUI, Jetpack Compose, WinUI/.NET, GTK/Qt.

- +Massime prestazioni e integrazione
- +Accesso a tutto l’hardware
- –Un codice per piattaforma

Tesi del corso

Nel 2026 il costo di “un codice per piattaforma” è crollato grazie agli agenti di programmazione. Vale la pena ripensare la scelta.

Il browser come piattaforma

Cosa sa fare oggi una web app (senza plugin):

- Lavorare **offline** (Service Worker, Cache API, IndexedDB)
- Ricevere **notifiche push**
- Accedere a **fotocamera, microfono, GPS**
- Leggere/scrivere file locali (File System Access API)
- Bluetooth, USB, seriale (con limiti e permessi)
- Grafica 3D e calcolo (WebGL, **WebGPU**)
- Codice compilato ad alte prestazioni (**WebAssembly**)
- Installazione "come app" (PWA)
- Condivisione nativa (Web Share), pagamenti (Payment Request)

Riferimento: `https://whatwebcando.today` e `https://caniuse.com`

Cosa NON sa fare (bene) il browser

- Girare in background quando la scheda è chiusa (a parte push e sync limitati).
- Accedere liberamente al file system, ai processi, alla rete a basso livello.
- Integrarsi con il sistema: barra di sistema, menu nativi, scorciatoie globali, widget della home.
- Garantire prestazioni costanti su hardware modesto (grafica complessa, liste enormi).
- Essere trovato nello store: su iOS la distribuzione fuori dallo store è di fatto limitata.

Da ricordare

Ognuno di questi limiti è il motivo per cui esistono Electron, Tauri, Capacitor e le app native. Li incontreremo uno per uno.

Come si sceglie: le domande giuste

1. Chi sono gli utenti e **dove** usano l'app? (negozio, magazzino, casa, in mobilità)
2. Serve funzionare **offline**? Quanto spesso?
3. Servono **capacità hardware** specifiche? (fotocamera, lettore di codici a barre, stampante etichette)
4. Serve essere **trovati** da Google? (allora è un sito, non un'app)
5. Quanto conta il **feeling nativo**? (vetrina per i clienti vs gestionale per i commessi)
6. Quante persone manterranno il codice tra due anni?

Esercizio da fare in aula su minicommerce: rispondete alle sei domande per il *catalogo* (clienti) e per l'*area admin* (negozio). Vengono fuori due risposte diverse: è il punto di partenza di tutto il corso.

Strumenti che useremo

- **Node.js** 24 LTS (o superiore): obbligatorio per il backend
- **npm**, incluso in Node: nessuna installazione in più
- **VS Code** con estensioni: Oxc, ESLint, Error Lens, GitLens, Tailwind CSS IntelliSense
- **Git** + account GitHub
- Browser: Chrome/Edge (DevTools) e Firefox
- Rust toolchain (per Tauri), Android Studio, Xcode (solo su Mac)

```
# Verifica installazione
node --version      # v24.x o superiore
npm --version       # 11.x
git --version

# Il progetto
git clone <url>/its2026_frontend_dev
cd its2026_frontend_dev
```

La toolchain moderna in breve

Vite	Dev server istantaneo e bundler di produzione. È quello che fa partire il frontend di minicommerce.
TypeScript	JavaScript con i tipi. In questo progetto **tutto** è TypeScript, anche il backend.
Oxlint	Il linter del progetto: scritto in Rust, trova errori e cattive pratiche in millisecondi. ESLint fa le stesse cose, più lentamente e con più plugin disponibili.
Prettier	Formatta il codice in modo uniforme. Non è configurato nel progetto: nel modulo 6 decideremo se aggiungerlo.
Vitest	Test runner compatibile con Vite. Il progetto non ha test: li scriviamo noi nel modulo 6.
Drizzle	L'ORM del backend. Lo sfioriamo soltanto: a noi interessa l'API che espone.

Configurazione del frontend

```
// vite.config.ts del progetto
export default defineConfig({
  plugins: [react(), tailwindcss()],
  resolve: {
    alias: { '@': path.resolve(import.meta.dirname, './src') },   // import '@/lib/api'
  },
  server: {
    proxy: { '/api': 'http://localhost:3001' },   // niente CORS in sviluppo
  },
})
```

- Il **proxy** fa sì che il browser veda una sola origine (`localhost:5173`): le chiamate ad `/api` vengono inoltrate al backend.
- In produzione il proxy non esiste: sarà il nostro primo problema di deploy (modulo 1) e tornerà su desktop e mobile.
- L'alias `@` evita i `../../..` negli import.

Che cos'è un package manager

- Il programma che **scarica e organizza le librerie** di cui il progetto ha bisogno, con tutte le loro dipendenze.
- Due file, una cartella e un registro:
  - `package.json`: cosa volete e in che versione
  - `package-lock.json`: cosa avete ottenuto *esattamente*
  - `node_modules/`: il codice scaricato
  - il **registro** (`registry.npmjs.org`): da dove si scarica
- Serve anche a **eseguire gli script** del progetto (`dev`, `build`, `lint`).

In questo corso: npm

È incluso in Node, quindi c'è già ovunque, anche sui PC del laboratorio dove non possiamo installare altro. minicommerce arriva con il suo `package-lock.json`: lo usiamo così com'è.

Da sapere

Esistono **pnpm**, **Yarn** e **Bun**. Installano gli *stessi* pacchetti dallo *stesso* registro: cambia come li mettono su disco. Ne parliamo fra due slide.

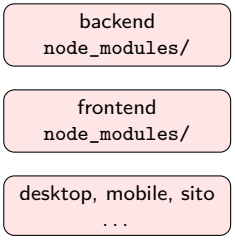
I comandi che useremo

Comando	Cosa fa
`npm install`	installa tutto quello che dice `package.json`
`npm install axios`	aggiunge una dipendenza e la scrive nel file
`npm install -D vitest`	dipendenza di sviluppo (non finisce in produzione)
`npm uninstall axios`	rimuove una dipendenza
`npm run dev`	esegue uno script di `package.json`
`npm run`	elenca gli script disponibili: usatelo in un progetto nuovo
`npx playwright test`	esegue un binario installato, o lo scarica al volo
`npm ci`	in CI: installa *esattamente* il lockfile, da zero
`npm audit`	vulnerabilità note nelle dipendenze

Il **lockfile va sempre committato**: è ciò che fa sì che voi, il compagno di banco e il server di build installiate le stesse identiche versioni.

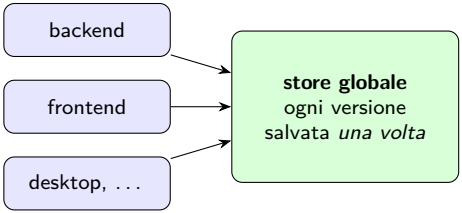
Il prezzo di npm: `node_modules`

- Ogni progetto riscarica e ricopia tutto: React, Vite e compagnia stanno **una volta per progetto**.
- minicommerce ha già **due** `node_modules`; nel corso diventeranno cinque o sei, con desktop, mobile e sito.
- npm appiattisce l'albero: in `node_modules/` finiscono anche le dipendenze *delle* dipendenze, e voi potete importarle per sbaglio.
- Funziona da voi, si rompe in CI o sul PC del compagno: sono le **phantom dependencies**.



Convivere con questi difetti si può: basta conoscerli. Importate solo ciò che avete dichiarato, e in CI usate `npm ci`.

L'alternativa che troverete: pnpm



- Ogni versione di ogni pacchetto è scaricata **una volta per macchina**; nei progetti arriva come **hard link**, non come copia.
- `node_modules/` contiene **solo** ciò che avete dichiarato: le phantom dependencies non esistono proprio.
- Non esegue gli script di installazione dei pacchetti senza permesso: una difesa in più (modulo 6).
- Yarn è storico e oggi di nicchia; Bun è un runtime oltre che un package manager, velocissimo, ancora poco diffuso in azienda.

Perché non lo usiamo: sui PC del laboratorio non possiamo installare nulla. Non è una bocciatura di pnpm, è un vincolo dell'aula.

Se un giorno volete passare a pnpm

```
# sul vostro computer, dove potete installare: una volta sola
npm install -g pnpm            # oppure: corepack enable
# senza installare nulla, per provarlo: npx pnpm --version

# conversione di un progetto esistente, dentro la cartella del progetto
npx pnpm import                # legge package-lock.json -> pnpm-lock.yaml
rm -rf node_modules package-lock.json
pnpm install

# da li' in poi
pnpm dev                       # al posto di npm run dev
pnpm add axios                 # al posto di npm install axios
```

- Il campo `"packageManager"` in `package.json` fissa lo strumento per chiunque apra il repository.
- Nel mondo del lavoro troverete tutti e quattro. Saper leggere un `package.json` e un lockfile conta più dello strumento.

I tipi del progetto

```ts
// frontend/src/lib/api.ts -- questi tipi li useremo per tutto il corso
export interface Category {
  id: number;
  name: string;
  description: string;
  productCount: number;      // calcolato dal backend
}

export interface Product {
  id: number;
  name: string;
  description: string;
  categoryId: number;
  category: string;          // nome della categoria, dalla join lato backend
  brand: string;
  price: number;
  stock: number;
  imageUrl: string | null;  // puo' mancare: il tipo lo dice
  createdAt: string;
}
```

Tipi derivati e union

```ts
// In scrittura la categoria si indica con l'id, e non servono id/createdAt
export interface ProductInput {
  name: string; description: string; categoryId: number;
  brand: string; price: number; stock: number; imageUrl: string | null;
}
// equivalente, senza riscrivere i campi:
type ProductInput2 = Omit<Product, 'id' | 'category' | 'createdAt'>;

// Union di stringhe: i valori ammessi sono solo questi
export interface ProductFilters {
  search?: string;                                  // ? = opzionale
  category?: string;
  minPrice?: string;
  maxPrice?: string;
  sort?: 'name' | 'price' | 'stock' | 'createdAt';
  order?: 'asc' | 'desc';
}
type SortField = NonNullable<ProductFilters['sort']>;   // 'name' | 'price' | ...
```

`Omit`, `Pick`, `Partial`, `NonNullable`: derivate un tipo da un altro invece di duplicarlo.

Narrowing e generics

```
function etichettaOrdinamento(s: SortField): string {
  switch (s) {
    case 'name':      return 'Nome';
    case 'price':     return 'Prezzo';
    case 'stock':     return 'Disponibilita';
    case 'createdAt': return 'Piu recenti';
    // se aggiungo un campo ordinabile e dimentico un case, TS mi avvisa
  }
}

// Generics: una funzione che vale per qualsiasi tipo
function groupBy<T, K extends string>(items: T[], key: (t: T) => K) {
  const out = {} as Record<K, T[]>;
  for (const it of items) (out[key(it)] ??= []).push(it);
  return out;
}
const perCategoria = groupBy(products, p => p.category);   // Record<string, Product[]>
```

Il client HTTP del progetto

```
import axios from 'axios';

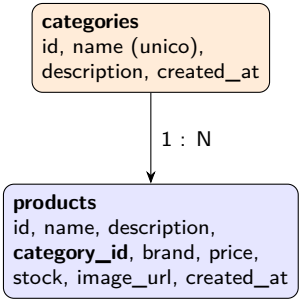
export const api = axios.create({ baseURL: '/api' });

// A ogni richiesta allega il token, se c'e'
api.interceptors.request.use((config) => {
  const token = localStorage.getItem(TOKEN_KEY);
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export async function fetchProducts(filters: ProductFilters = {}) {
  const { data } = await api.get<Product[]>('/products', { params: filters });
  return data;
}
```

- **Axios** lancia un'eccezione sugli status 4xx e 5xx; `fetch` no: lì si controlla `res.ok`.
- `params` costruisce la query string: `/products?search=mouse&sort=price`.
- Un unico punto di ingresso per tutte le chiamate: base URL, token, errori.

Il modello dati



- Una categoria raggruppa molti articoli; ogni articolo appartiene a **una** categoria.
- Chiave esterna con `ON DELETE RESTRICT`: una categoria con articoli **non** si può cancellare.
- C'è anche `users` (email, password_hash, name, is_admin) per il login.
- Le risposte sugli articoli contengono sia `categoryId` sia `category` (il nome): niente seconda chiamata per l'etichetta.

Gli endpoint

Metodo	Rotta	Auth	Note
POST	`/api/auth/login`	–	`{email, password}` → `{token, user}`
POST	`/api/auth/register`	–	crea un cliente
GET	`/api/categories`	–	categorie con `productCount`
GET	`/api/products[/:id]`	–	lista con filtri, oppure il dettaglio
POST / PUT / DELETE	`/api/products[/:id]`	admin	crea, aggiorna, elimina
GET	`/api/cart`	cliente	`{ items, total }`
POST / PATCH / DELETE	`/api/cart/items[/:id]`	cliente	aggiunge, cambia quantità, rimuove
POST	`/api/orders`	cliente	ordine finto: accetta e svuota il carrello

Filtri del catalogo: `search`, `category`, `categoryId`, `brand`, `minPrice`, `maxPrice`, `sort`, `order`. Header `Authorization: Bearer <jwt>`: senza token 401, con il ruolo sbagliato 403. Il **carrello vive sul server**, legato all'utente: non è uno stato del browser.

Esempio di richiesta e risposta

```
GET /api/products?search=mouse&sort=price&order=asc
200 OK
[
  {
    "id": 7,
    "name": "Mouse wireless Pro",
    "description": "Mouse ergonomico 2.4 GHz con ricevitore USB-C",
    "categoryId": 2,
    "category": "Periferiche",
    "brand": "Logitech",
    "price": 39.9,
    "stock": 18,
    "imageUrl": null,
    "createdAt": "2026-09-20 09:12:00"
  }
]
```

- `price` è un numero (`real` in SQLite): la formattazione in euro è compito del frontend.
- Gli id sono numeri interi assegnati dal database.

Provare l'API senza il frontend

```
# elenco articoli
curl http://localhost:3001/api/products | head

# login: prendiamo il token
curl -X POST http://localhost:3001/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"admin@minicommerce.dev","password":"admin123"}'

# creare un articolo (serve il token)
curl -X POST http://localhost:3001/api/products \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"name":"Tastiera 60%","categoryId":2,"brand":"Keychron","price":89.9,"stock":5}'
```

- Imparate a leggere l'API **prima** di scrivere il frontend: metà dei bug sono malintesi sul contratto.
- Alternative a curl: l'estensione REST Client di VS Code, Bruno, Postman, o la scheda Rete del browser.

Git durante il corso

- Un repository per studente, partendo dal codice di minicommerce che vi consegno.
- Un branch per laboratorio: `lab/01-query`, `lab/02-tauri`, ...
- Commit piccoli con messaggi che spiegano il *perché*.
- Pull request verso `main` a fine laboratorio: la userò per la valutazione continua.
- Mai committare segreti: il `JWT_SECRET` di produzione non entra nel repository.
- Il file del database (`minicommerce.db`) è locale: non si committa, si rigenera con `db:seed`.

Convenzione per i messaggi

`feat(catalogo): filtro per marca` – `fix(auth): gestione del 401` – `chore: aggiorna dipendenze`

Agenti di programmazione: le regole della classe

- Potete usare Copilot, Claude Code, Cursor, Codex e simili: nel 2026 fa parte del mestiere.
- Nel repository c'è già un `AGENTS.md` con le convenzioni del progetto: leggetelo, è anche il vostro manuale di stile.
- Regola 1: **dovete saper spiegare** ogni riga che consegnate. Se non la capite, non è vostra.
- Regola 2: l'agente scrive, **voi rivedete**. Il diff si legge prima di committare.
- Regola 3: nei laboratori marcati "a mano" l'agente resta chiuso: servono a costruire le basi.
- Regola 4: nel progetto finale documentate *come* avete usato l'agente (prompt, cosa ha sbagliato).

Laboratorio 0

Laboratorio: Avviare e capire minicommerce

1. Clonare il repository, avviare backend (`db:push`, `db:seed`, `dev`) e frontend; entrare come admin.
2. Provare l'API con curl o REST Client: lista articoli, login, creazione di un articolo.
3. Disegnare su carta il percorso di un dato: dal click sul filtro "Periferiche" alla card sullo schermo, passando per i file toccati.
4. Prima modifica: aggiungere al catalogo il filtro per **marca** (il backend lo supporta già, il frontend no), poi aprire una PR.

Domande di verifica

Perché in sviluppo non serve configurare CORS? Che differenza c'è tra `categoryId` e `category` nella risposta? Dove viene aggiunto il token alle richieste? Cosa succede se cancellate una categoria che ha articoli?